

Architectural Foundations for Orchestrator-Driven, Local-First Creative AI Runtimes

The development of high-performance creative software is transitioning from traditional monolithic widget hierarchies to decentralized, orchestrator-driven architectures. This evolution is necessitated by the emergence of creative AI runtimes that must seamlessly integrate heterogeneous rendering modes—ranging from 2D vector graphics and complex typography to real-time neural radiance fields—while maintaining the strict latency requirements of professional creative tools. A kernel-like UI core, prioritizing message-passing and capability-based routing, offers a robust framework for managing these diverse primitives. The following report synthesizes modern research (2020–2026) to define the state of the art in UI composition, GPU rendering, and local-first architecture, providing a technical blueprint for a Rust-based creative runtime.

UI Architecture and Composition

The design of a composition orchestrator requires a fundamental departure from the rectangular buffer management typical of display servers like Wayland. Instead, a kernel-like architecture treats rendering primitives as isolated services that communicate via a central message bus. This approach draws significant inspiration from microkernel design, where inter-process communication (IPC) and resource isolation are the primary constraints for system stability and performance.

Heterogeneous Rendering and Pluggable Architectures

Modern research into heterogeneous rendering composition focuses on the ability to merge disparate visual representations into a single coherent frame. Recent SIGGRAPH contributions highlight the shift toward content-adaptive image representations, such as Image-GS, which utilizes 2D Gaussians to balance visual fidelity with memory footprint. In a composition-first UI, such representations allow the orchestrator to route specific render intents to specialized GPU-compute nodes rather than relying on a global rasterization pass.

Research into material layering further extends this concept. By performing layering in the "neural space" through compressed Bidirectional Reflectance Distribution Functions (BRDFs) and latent vectors, systems can composite complex surfaces—such as metallic car paint or multi-layered decorative textures—with physically based accuracy while maintaining real-time performance. For a creative runtime, this implies that the UI orchestrator must handle not just pixels, but high-dimensional data representing material properties and lighting intents.

The maturity of these technologies varies. While compute-based 2D rendering is nearing production readiness, neural-space layering remains largely in the research stage, limited by the high VRAM requirements of current generation GPUs. However, the "bus-first" architecture is ideally suited to this trajectory, as it allows new neural rendering primitives to be plugged into the

system as capabilities as hardware evolves.

Composition Models and Display Server Research

The limitations of X11 and Wayland in handling high-precision creative workflows have led to research into novel compositor architectures. Beyond simple surface management, the core challenge is ensuring temporal isolation—preventing a heavy 3D primitive from causing jitter in the text-entry primitive. Microkernel research on seL4 demonstrates that priority and budget awareness in IPC processing are critical to preventing "Thundering Herd" effects, where high-priority threads (like the UI cursor) are delayed by lower-priority tasks (like a background AI generation).

Novel approaches to compositing move beyond the hierarchical surface model toward a flat, capability-based routing system. In this model, primitives advertise their capabilities—such as "VectorRasterizer" or "NeuralShadowing"—and the orchestrator routes work packages to them. This mirrors the SCION router's design for high-assurance network routing, which emphasizes isolation and failure control.

Composition Paradigm	Core Mechanism	Routing Strategy	Maturity (2026)
Traditional (Wayland)	Buffer Swapping	Hierarchical Z-Order	Production
Bus-Based (Actor)	Message Passing	Priority-Aware IPC	Early Adoption
Capability-Based	Capability Tokens	Resource-Granular Routing	Research-Stage
Neural Hybrid	Latent Space Layering	Differentiable Composition	Prototype

Retained vs. Immediate Mode Hybridization

The Rust ecosystem has seen a convergence between the ease of use of immediate-mode (IMUI) and the performance of retained-mode UI. Research into statically typed view trees, exemplified by the Xilem project, provides a compelling middle ground. In Xilem, lightweight view objects are recreated on every interaction, but a reconciliation pass updates a persistent widget tree. This ensures that the orchestrator only routes "delta" messages across the bus, rather than recreating the entire scene graph.

The architectural choice of using statically typed view trees over a dynamically typed Virtual DOM (VDOM) is significant. VDOMs, common in web-based architectures, incur overhead due to hash-map allocations and dynamic type checking. Conversely, a statically typed tree allows the compiler to know exactly which fields have changed, reducing the diffing process to a minimal set of memory comparisons. For a professional creative tool, this reduces CPU jitter during complex UI updates.

GPU Rendering for UI

GPU-accelerated 2D rendering has evolved from simple hardware-assisted rasterization to complex compute-centric pipelines. In the context of a Rust-based orchestrator, the most relevant work focuses on bypassing traditional graphics APIs (like OpenGL) in favor of massively parallel compute shaders.

State of the Art in Vector Rendering

The "Pathfinder" lineage has largely been superseded by compute-centric renderers such as Vello and Forma. Vello, developed as part of the Linebender project, utilizes a 9-stage GPU compute pipeline to handle complex vector geometry without the need for traditional CPU-side tessellation. This pipeline handles everything from path clipping and masking to complex gradients directly in shaders, significantly reducing the bottleneck of transferring geometry data from the CPU to the GPU.

Vector Engine	Primary Rendering Path	GPU Requirement	Key Innovation
Vello	GPU Compute (9-stage)	Tier 3 (Vulkan/Metal)	Direct path rasterization via compute
Forma	CPU/GPU Hybrid	Moderate	Balanced workload distribution
tiny-skia	CPU Rasterization	None	High-fidelity fallback/WASM focus
Impeller	GPU Tessellation	Tier 2/3	Used in Flutter; optimized for mobile

A critical innovation in 2026 is the integration of Lottie animations via projects like Velato, which allows complex motion graphics to be treated as native Vello primitives. For a creative AI runtime, this means the UI can handle live, vector-based AI visualizations with the same performance as static icons.

Surface Management and Multi-Source Mixing

A composition orchestrator must manage surfaces derived from diverse sources, such as a 3D preview, a vector UI layer, and a neural generation canvas. Tools like surfman provide the necessary cross-platform surface management primitives, facilitating the movement of GPU textures between different contexts without redundant copies.

Research into "neural shadowing" provides a mechanism for adding high-quality soft shadows and lighting to these mixed surfaces. By Specializing the neural architecture to be memory bandwidth-aware, these techniques produce results on par with ray tracing but at the speeds required for interactive UI. This allows the orchestrator to composite a 2D vector element "above" a 3D scene with realistic contact shadows, bridging the gap between flat UI and spatial content.

Latency and Frame Timing Optimization

Professional creative software requires extreme precision in input-to-photon latency. Research into "Late Latching" demonstrates that providing the GPU with the most current input poses (like stylus position) at the very last moment before rendering begins can eliminate up to one full frame of latency. While originally developed for Meta Quest XR applications, this technique is increasingly relevant for high-DPI stylus input in creative tools, where a 16ms delay can be perceived as a disconnect between the tool and the canvas.

Measurement techniques have matured, moving from simple software-based logging to high-speed camera analysis that accounts for motion prediction. This research indicates that while sudden movements can spike latency to 42ms, predictive algorithms can reduce this to as little as 2ms once the motion pattern is established. The UI orchestrator should therefore treat

input prediction as a core routing capability rather than a peripheral feature.

Text Rendering and Layout

Text rendering is a non-negotiable requirement for creative software, requiring support for complex scripts, variable fonts, and high-quality hinting. In a non-browser Rust environment, this has traditionally been a point of friction, but recent developments have created a robust, native stack.

Evolution of the Rust Text Stack

The competition and eventual convergence between the Parley and Cosmic-text projects define the current landscape. Parley is focused on high-performance algorithms and variable font support, while Cosmic-text provides a more complete solution for multi-line rich text and editing. Both libraries leverage swash for rasterization, which research indicates provides superior hinting and sub-pixel rendering compared to traditional libraries like FreeType.

Component	Industry Standard (C++)	Rust Alternative (2026)	Technical Focus
Shaping	HarfBuzz	rustybuzz / HarfBuzz (Rust)	Complex script support/bidi
Rasterization	FreeType	swash / skrifa	Hinting and sub-pixel quality
Layout	ICU/DirectWrite	parley / cosmic-text	Performance and Unicode compliance
Font Loading	Fontconfig	font-kit / fontations	Platform-agnostic font discovery

The migration of Bevy from Cosmic-text to Parley highlights the industry's demand for variable font support and better integration with GPU-compute backends like Vello. For an orchestrator-driven UI, text is no longer rendered as a separate bitmap and blitted; instead, it is often treated as a set of vector paths that are directly rasterized by the GPU in the final composition pass.

Typography in Non-Browser Contexts

Professional creative tools require typography that matches the rendering quality of their primary content. This includes support for locale-based word segmentation and AccessKit integration for accessibility. The challenge for the orchestrator is caching these results efficiently. Research suggests that while rendering full strings as bitmaps (the "Cosmic-text" approach) improves anti-aliasing, it introduces significant texture-switching overhead. A "bus-first" approach favors streaming glyph info as a set of draw calls to the GPU, allowing the orchestrator to manage glyph caching at the level of the GPU-compute unit.

Rust UI Ecosystem Technical Comparison

The Rust UI ecosystem is characterized by a "Cambrian explosion" of frameworks, each making distinct architectural trade-offs. For a creative AI runtime, the choice depends on the degree of

control required over the rendering pipeline.

GPUI: The Application-First Framework

GPUI, the foundation of the Zed editor, is notable for being built for a practical, non-trivial use case. It utilizes a GPU-native approach via the Blade graphics abstraction layer, which supports Metal, Vulkan, and GLES. GPUI's architecture is inspired by Tailwind CSS, providing a declarative styling system that is highly performant. A key development in 2026 is the "GPUI Community Edition" (gpui-ce), a fork intended to provide a more general-purpose framework as the original project focuses strictly on Zed's requirements.

Xilem and the Research Path

Xilem represents a major investment in research-led UI design. It avoids the performance pitfalls of standard VDOM architectures by using statically typed view trees that are updated via a reconciliation pass. This architecture is particularly well-suited for the orchestrator-driven model, as it provides a clear separation between the "view" (the message sent to the orchestrator) and the "widget" (the rendering state managed by the orchestrator).

Servo and Verso: Viability for Embedding

As of 2026, Servo has seen a resurgence as a viable engine for embedding. It passes 79% of WPT tests and includes support for WebGPU, which is critical for creative AI workloads that require browser-based UIs to access high-performance compute. Verso, a browser project built on Servo, is actively expanding into a stable webview library for embedding purposes, offering a potential bridge for tools that need to mix standard web content with native creative primitives.

Comparative Framework Table

Framework	Architecture	State Management	Rendering	Maturity (2026)	:---	:---	:---	
:---	:---	GPUI	Entity-Component	Imperative Views	Blade (GPU)	Production	Xilem	
Typed Reactive	Reconciled Tree	Vello (Compute)	Beta/Research	Iced	Elm Architecture	Message Passing	WGPU	Production
Slint	Model-View	Property Binding	Software/GPU	Mature (Embedded)	Dioxus	VDOM-based	Signals/Hooks	Webview/WGPU
Production	Floem	Reactive	Fine-grained	Vello	Early Adoption			

The honest assessment of the ecosystem is that while GPUI and Iced are mature enough for production, Xilem and Floem offer the most innovative ideas for a next-generation orchestrator-driven runtime. Dioxus and Tauri remain the preferred choices for applications that can tolerate the overhead of the web stack.

Alternative UI Paradigms and Creative Tools

Professional creative software is moving toward "infinite canvas" and node-based paradigms, which fundamentally change the requirements of the UI orchestrator.

Node-Based and Dataflow Paradigms

The rise of AI has accelerated the adoption of node-based workflows. Tools like NodeTool and Krea Nodes provide infinite canvases where each node represents an AI model call, a data transformation, or a rendering task. The architecture of these tools is naturally "bus-first," as the edges between nodes represent message routes.

Figma’s acquisition of Weavy (now Figma Weave) in 2025 signaled a shift toward professional-grade node platforms that bridge the gap between AI generation and precision design. These platforms prioritize repeatable, high-fidelity results and allow workflows to be converted into simplified "app mode" interfaces for non-technical collaborators.

Visual Programming in Creative Contexts

Research into "programmable ink" and malleable software from Ink & Switch's Tenfold project suggests a future where the UI is not a static product but a "workshop" of tools. Tenfold utilizes a "playground"—a tool for making tools—which allows users to build complex visualizations, like catenoids or Lissajous waves, through a visual control surface. For a creative runtime, this implies that the UI orchestrator must support dynamic reconfiguration of its rendering primitives at runtime, allowing the user to "wire up" new UI capabilities as needed.

Local-First and Architecture Philosophy

A creative AI runtime must be local-first to ensure data ownership and zero-latency interaction. This requires rethinking the entire software stack, from the database to the access control layer.

Conflict-Free Replicated Data Types (CRDTs)

CRDTs are the foundational technology for local-first synchronization. Libraries like Automerge and Yjs have matured significantly, with Automerge v3.0 offering 10x memory reduction and high-performance binary formats. These allow the UI state (e.g., the position of a node, the text in a buffer) to be synchronized across devices without a central server.

Convergent Capabilities and Decentralized Access Control

A major breakthrough in 2024–2025 was the development of Convergent Capabilities (Concap) by the Ink & Switch team. Unlike traditional object-capabilities that fail during network partitions, Concap integrates CRDT state to allow authority to flow even in offline environments.

Auth Model	Offline Support	Coordination	Primary Use Case
Object-Cap	No	synchronous	Kernel-level security
Cert-Cap (UCAN)	Yes	stateless	Web3/Decentralized apps
Concap (Keyhive)	Yes	concurrent	Local-first collaboration
TreeKEM (Beehive)	Yes	group-based	E2EE for large groups

The Beehive project further extends this by exploring TreeKEM-based encryption for local-first protocols, providing forward-secure key agreement for large groups of users. This ensures that "identity-aware" computing—where the system knows about user context and brand voice—remains secure and private within the user's local environment.

Performance, Latency, and Kernel Design Patterns

To reach professional standards, a UI orchestrator must adopt the rigorous latency controls of high-frequency trading and operating system kernels.

Hot Path vs. Cold Path Separation

Applying kernel design patterns, the UI must separate the "Hot Path" of synchronous user interactions from the "Cold Path" of background persistence and AI inference. In high-performance financial systems, this is achieved by pinning critical threads to specific CPU cores and offloading logging and I/O to asynchronous ring buffers.

Shopify's architecture provides a scalable model: the Hot Path handles synchronous, user-facing operations (like a checkout or, in this case, a brush stroke), while the Cold Path asynchronously dispatches "OrderCreated" events for fulfillment. In a UI runtime, the orchestrator should handle the mouse-to-GPU pipeline on the Hot Path, while AI model weights and sync protocols reside on the Cold Path, ensuring that network latency never blocks the UI thread.

Latency Reduction and Predictability

Reducing latency is a matter of minimizing network hops and data serialization costs. Research in trading systems architecture emphasizes the use of SBE (Simple Binary Encoding) for direct memory access without parsing, which is a pattern the Rust ecosystem's rkyv or zerocopy crates can fulfill.

Predictable, low-jitter performance also requires a "clean-slate" design for inter-component communication. The SCION router project demonstrates that by building failure isolation and route control into the core architecture, systems can achieve microsecond-range predictability even under heavy load.

Adjacent Fields: Lessons from Games and Pro Tools

The convergence of game engines and creative software provides a final set of insights for the orchestrator design.

Game Engine UI Architectures

Game engines like Bevy have pioneered the use of Entity-Component Systems (ECS) for UI, allowing for massive parallelism in layout and styling. However, research suggests that for non-game applications, the "statically typed view tree" of Xilem may be more efficient for complex, deeply nested UIs than a flat ECS. The game industry's focus on "Late Latching" and "Motion-to-Photon" (MTP) measurement is the gold standard for UI responsiveness that the creative AI runtime must meet.

Professional Creative Tool Structures

Professional tools like Houdini and DaVinci Resolve manage heterogeneity by using "procedural systems" for form-finding and "nodes" for post-production. Their UIs are effectively visual

representations of high-performance DAGs (Directed Acyclic Graphs). This teaches us that the UI orchestrator should not just composite surfaces, but manage the execution order of the underlying dataflow, ensuring that "durable commits" (like saving a frame) do not block the interactive Hot Path.

Synthesis: A Blueprint for a Rust-Based UI Orchestrator

The intersection of these eight research areas reveals a clear architectural path for the proposed local-first creative AI runtime.

The Orchestrator as a UI Microkernel

The core should be an extremely thin, high-performance message bus implemented in Rust. It does not possess knowledge of specific widgets; instead, it manages "capabilities" provided by pluggable primitives.

- **Routing and IPC:** Utilize a non-preemptive, priority-aware IPC mechanism inspired by seL4 to prevent priority inversion between a heavy AI generation task and the UI cursor.
- **Rendering Primitives:** Standardize on Vello for 2D vector work and WGPU for 3D/AI surfaces. Use surfman for cross-platform surface management and "Late Latching" for stylus/mouse input.
- **State and Sync:** Use Automerge (v3.0) for the UI state graph, ensuring every UI element is a CRDT node. Implement Convergent Capabilities for decentralized access control.
- **Text and Typography:** Integrate Parley with a GPU-compute-centric glyph cache to treat text as a high-fidelity vector primitive.
- **Performance:** Strictly separate the Hot Path (input/render/composition) from the Cold Path (AI inference/persistence/sync) using asynchronous ring buffers and pinned cores.

Maturity Assessment of Proposed Components

Component	Technology	Maturity	Action
Vector Engine	Vello	Mature	Use as primary 2D renderer
UI State	Automerge	Mature	Foundation for local-first sync
Auth/Identity	Keyhive/Concap	Research	Implement for collaborative modes
Layout	Parley/Xilem-style	Mid-Maturity	Adopt for professional-grade UI
Embedding	Servo/Verso	Rapidly Evolving	Use for secondary/web-based UI
Neural Rendering	Image-GS	Research	Design as a future "capability"

By adopting this orchestrator-driven approach, the creative AI runtime can achieve a "workshop" feel—where tools are malleable, data is owned by the user, and interaction is instantaneous.

The architectural mirroring of the system's core as a message-passing, capability-based kernel ensures that as AI rendering technologies evolve, the UI can adapt without requiring a foundational rewrite. This approach moves UI from being a static facade to a dynamic, high-performance engine for creative expression.

Works cited

1. The Thundering Herd: Amplifying Kernel Interference to Attack Response Times - School of Engineering & Applied Science - The George Washington University, https://www2.seas.gwu.edu/~gparmer/publications/ras22sel4_attacks.pdf
2. Most Influential SIGGRAPH Papers (2026-03 Version) - Paper Digest, <https://www.paperdigest.org/2026/03/most-influential-siggraph-papers-2026-03-version/>
3. SIGGRAPH '22: ACM SIGGRAPH 2022 Conference Proceedings, <https://www.siggraph.org/wp-content/uploads/2022/08/SIGGRAPH-22-ACM-SIGGRAPH-2022-Conference-Proceedings.html>
4. NGI Zero Core - NLnet Foundation, <https://nlnet.nl/thema/NGIZeroCore.html>
5. Release Xilem 0.1.0 · linebender/xilem : r/rust - Reddit, https://www.reddit.com/r/rust/comments/1cmo1pt/release_xilem_010_linebenderxilem/
6. GitHub - gogpu/gg: Enterprise-grade 2D graphics library for Go. Pure Go, zero CGO. Part of GoGPU ecosystem., <https://github.com/gogpu/gg>
7. Graphics APIs — list of Rust libraries/crates // Lib.rs, <https://lib.rs/rendering/graphics-api>
8. Enable Late Latching | Meta Horizon OS Developers, <https://developers.meta.com/horizon/documentation/unreal/unreal-late-latching/>
9. Latency and Cybersickness: Impact, Causes, and Measures. A Review - Frontiers, <https://www.frontiersin.org/journals/virtual-reality/articles/10.3389/frvir.2020.582204/full>
10. Measuring motion-to-photon latency for sensorimotor experiments with virtual reality systems | Leeds Institute for Data Analytics, <https://lida.leeds.ac.uk/research-projects/motion-to-photon-latency-vr/>
11. questions about text rendering · Issue #452 · linebender/vello - GitHub, <https://github.com/linebender/vello/issues/452>
12. possible migration of `bevy\_text` from `cosmic-text` to `parley` · Issue #21765 · bevyengine/bevy - GitHub, <https://github.com/bevyengine/bevy/issues/21765>
13. Color Emoji support · Issue #2551 · emilk/egui - GitHub, <https://github.com/emilk/egui/issues/2551>
14. Google is rewriting HarfBuzz and FreeType in Rust - Reddit, https://www.reddit.com/r/rust/comments/1e0dfj6/google_is_rewriting_harfbuzz_and_freetype_in_rust/
15. Cosmic Text for font rendering #3378 - emilk/egui - GitHub, <https://github.com/emilk/egui/issues/3378>
16. gpui fork : r/rust - Reddit, https://www.reddit.com/r/rust/comments/1pl9lz0/gpui_fork/
17. Servo in 2024: stats, features and donations - Servo aims to empower developers with a lightweight, high-performance alternative for embedding web technologies in applications., <https://servo.org/blog/2025/01/31/servo-in-2024/>
18. Planning to switch to Rust for desktop development - Reddit, https://www.reddit.com/r/rust/comments/1lt24wx/planning_to_switch_to_rust_for_desktop_development/
19. NodeTool: Node-Based Visual Builder for AI Workflows and LLM Agents, <https://nodetool.ai/>
20. Top 7 node-based AI workflow apps in 2025 - Krea, <https://www.krea.ai/articles/top-node-based-ai-workflow-apps>
21. 2026: The Year of the Node-Based Editor | by Erik Fadiman | Medium, <https://medium.com/@fadimantium/2026-the-year-of-the-node-based-editor-941f0f15d467>
22. Tenfold - Ink & Switch, <https://www.inkandswitch.com/project/tenfold/story/>
23. alexanderop/awesome-local-first: Useful Links for Everything related to LocalFirst - GitHub,

<https://github.com/alexanderop/awesome-local-first> 24. A Local-First Case Study | jakelazaroff.com, <https://jakelazaroff.com/words/a-local-first-case-study/> 25. Keyhive: Local-first access control - Ink & Switch, <https://www.inkandswitch.com/keyhive/notebook/> 26. Local-First group- and message encryption in p2panda, <https://p2panda.org/2025/02/24/group-encryption.html> 27. Trading System Architecture 2026 | From Microservices to Agentic Mesh, <https://www.tuvoc.com/blog/trading-system-architecture-microservices-agentic-mesh/> 28. Event-Driven Architecture: Handling Surge Traffic in Microservices | by Om Lanke - Medium, <https://medium.com/@om.lanke/event-driven-architecture-handling-surge-traffic-in-microservice-s-992d14e0a2af> 29. From Agent OS to Agent Processor Unit | by EZEKIAS BOKOVE | Google Cloud - Medium, <https://medium.com/google-cloud/from-agent-os-to-agent-processor-unit-cdd797fb6287> 30. 15 Best 3D Modeling Software for Professionals (2026), <https://www.archivinci.com/blogs/best-3d-modeling-softwares> 31. Why Postgres wants NVMe on the hot path, and S3 everywhere else - The New Stack, <https://thenewstack.io/postgres-nvme-s3-storage/>